

THE CLAUDE BEAST

Claude Plan Mode

Read-Only Exploration for Safer, Smarter Development

Anthropic | Claude Code | 2026

What Is Plan Mode?

01

A built-in feature that separates research and analysis from code execution



- **Read-Only Mode** — Claude can read files, search code, and explore your project but cannot modify anything
- **Structured Plans** — Produces a markdown implementation plan written to disk
- **Safety First** — No accidental file edits, no unintended command execution
- **User Control** — Plans are reviewed, edited, and approved before any execution begins



The Plan File

- Standard markdown file in Claude's plans directory
- Claude uses Edit tool internally to write its own plan
- Configurable location via plansDirectory setting
- Can be reviewed and iterated before execution

When to Use Plan Mode

Best Use Cases



- Complex multi-file implementations
- Large-scale refactors and migrations
- Exploring unfamiliar repositories
- Dependency and impact analysis
- Architecture decision-making
- Sensitive production code changes

When to Skip It



- Quick single-file edits
- Boilerplate generation
- Interactive experimentation
- Typo corrections, single-line fixes
- Stable codebase with trusted patterns

Rule of thumb: Match the tool to the complexity. Plan for multi-file, skip for trivial.

Activation Methods

03

`Shift + Tab`

Keyboard Shortcut

Cycle through modes:
Normal > Auto-Accept >
Plan > Normal

`/plan`

Slash Command

Enter Plan Mode directly
from the prompt (v2.1.0+)

`--permission-mode
plan`

CLI Flag

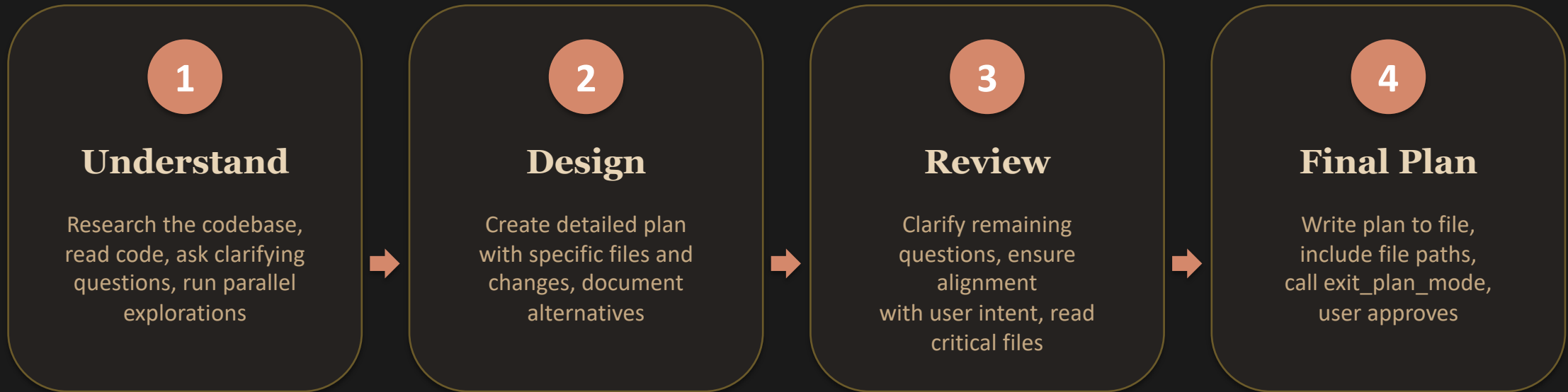
Launch Claude Code
in Plan Mode from terminal

`/model opus`

Model Command

Opus for planning,
Sonnet for execution

The 4-Phase Planning Workflow



After Phase 4: User reviews the plan, can edit the markdown directly, then exits Plan Mode to begin execution

The opusplan Model Alias

05

Opus-quality reasoning for planning, Sonnet-speed for execution

Plan Phase: Opus 4.6

- 1M context window for massive codebases
- Superior reasoning for architecture decisions
- Deep analysis of complex dependencies

Execute Phase: Sonnet 4.6

- Fast, cost-efficient code generation
- Handles implementation at lower cost
- Ideal for straightforward coding tasks

Subagent Architecture

Plan Subagent



- Dedicated subagent for planning tasks (v2.0.28+)
- Supports resumption and reuse across sessions
- Dynamic model selection per task

Explore Subagent



- Haiku-powered for efficient codebase searching
- Saves context tokens via lightweight model
- Auto-delegated for comprehensive exploration

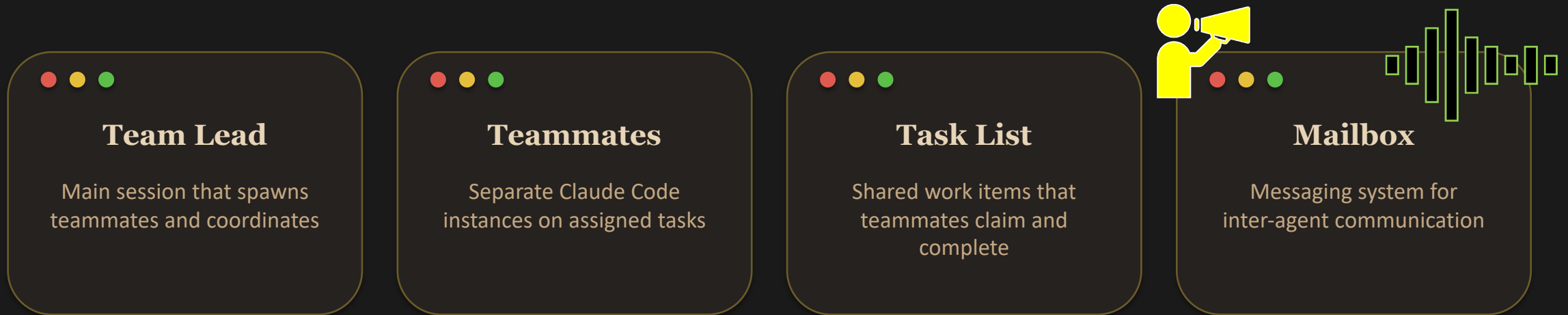
Built-in Agent Types

Agent Type	Purpose	Default Model
Explore	Fast codebase navigation	Haiku 4.5
General-Purpose	Complex multi-step tasks	Sonnet 4.5
Code-Reviewer	Code quality analysis	Sonnet 4.5
Security-Auditor	Vulnerability detection	Sonnet 4.5
Debugger	Root cause analysis	Sonnet 4.5

Agent Teams Integration

07

Multi-agent coordination extends Plan Mode's concepts (Feb 2026)



Plan Approval Flow for Teammates

- Teammates can be required to plan before implementing — "Require plan approval before changes"
- Teammate sends plan approval request to lead, who reviews and approves or rejects with feedback
- Best practice: Plan first with Plan Mode to define roles, then deploy Agent Teams for parallel execution

Superpowers Plugin Integration

08

Structured, enforced planning workflows on top of native Plan Mode (obra/superpowers)

Key Planning Commands



- **/superpowers:brainstorm** — Refine rough ideas before coding
- **/superpowers:write-plan** — Create detailed implementation plans
- **/superpowers:execute-plan** — Run plans in batches with checkpoints

Enforced Workflows



- **Test-Driven Development** — RED-GREEN-REFACTOR cycle
- **Systematic Debugging** — 4-phase root cause analysis
- **Verification Before Completion** — Must verify before done

Why Layer Superpowers on Native Plan Mode?

Persistent plans as markdown files survive across sessions | Token-efficient batch execution | Progress tracking on disk

Key Takeaways

1

Plan Before You Code

Plan Mode enforces read-only exploration, producing structured plans before any execution

2

Match Tool to Complexity

Use Plan Mode for multi-file work; skip it for simple edits

3

Optimize with opus

Opus-quality analysis + Sonnet-speed execution = best of both worlds

4

Scale with Agent Teams

Plan first, then deploy parallel teammates for complex projects

5

Extend with Superpowers

Layer enforced workflows, persistent plans, and batch execution on top

The Claude Beast | Plan Mode | Think first, code second.